

Лабораторна робота №8.

Управління багатозадачною системою за допомогою ІЧ-пульта

Мета роботи. Навчитися приймати та обробляти сигнали від інфрачервоного пульта дистанційного керування та реалізувати вибір задач або режимів роботи системи за допомогою ІЧ-пульта.

Теоретичні відомості.

Прийом сигналів від ІЧ-пульта (інфрачервоного пульта дистанційного керування) організовується через ІЧ-приймач, який перетворює інфрачервоні імпульси, передані світлодіодом пульта, у цифрові сигнали, що можуть бути зчитані мікроконтролером.

Більшість ІЧ-пультів використовують протокол NEC, у якому передача команди відбувається через послідовність імпульсів, закодованих за допомогою модуляції частотою 38 кГц.

Схематично сигнал має таку структуру:

Елемент	Тривалість	Опис
Start pulse	~9 мс HIGH + 4.5 мс LOW	Початок пакета
Data bits (32 біти)	кожен біт – 562 мкс HIGH + (562 або 1687 мкс LOW)	Коди даних
Stop pulse	562 мкс HIGH	Кінець пакета

Кожна команда складається з 32 біт: 8 біт – адреса пристрою, 8 біт – інверсія адреси, 8 біт – команда (код кнопки), 8 біт – інверсія команди.

Модуль приймає ІЧ-світло з довжиною хвилі ~940 нм і відфільтровує несучу частоту 38 кГц, видаючи вже готовий цифровий логічний сигнал (0/1) на вихід. При відсутності сигналу – вихід у стані логічної 1. Під час прийому імпульсу – вихід короткочасно переходить у логічний 0. Тобто на GPIO лінію Pico надходить послідовність імпульсів, які кодують бінарну послідовність (0 та 1).

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Завантажте в каталог вашого проєкту бібліотеки для роботи з OLED дисплеєм, сенсорами та ІЧ-пультом перейшовши за посиланням:

<https://github.com/mpavlyk/rtos/blob/main/lib/lib.zip>

3. Відкрийте файл **CMakeLists.txt**, який знаходиться у каталозі вашого проєкту і відредагуйте його наступним чином:

- знайдіть рядок із інструкцією `pico_sdk_init()`. Після цієї інструкції вставте:

```
1 # Add Subdirectory
2 add_subdirectory(/lib/Config)
3 add_subdirectory(/lib/OLED)
4 add_subdirectory(/lib/Fonts)
5 add_subdirectory(/lib/GUI)
6 add_subdirectory(/lib/SHTC3)
7 add_subdirectory(/lib/SGP40)
8 add_subdirectory(/lib/QMI8658)
9 add_subdirectory(/lib/nec_receive_library)
10
11 # Include Directories
12 include_directories(/lib/Config)
13 include_directories(/lib/OLED)
14 include_directories(/lib/Fonts)
15 include_directories(/lib/GUI)
16 include_directories(/lib/SHTC3)
17 include_directories(/lib/SGP40)
18 include_directories(/lib/QMI8658)
19 include_directories(/lib/nec_receive_library)
```

- Знайдіть інструкцію - `target_link_libraries(lab01 pico_stdlib)` і додайте бібліотеки - `hardware_spi`, `hardware_i2c`, `hardware_pwm`, `hardware_adc`, `Config`, `GUI`, `Fonts`, `OLED`.

```
1 target_link_libraries(lab
2     pico_stdlib
3     FreeRTOS-Kernel
4     FreeRTOS-Kernel-Heap4
5     hardware_spi
6     hardware_i2c
7     hardware_pwm
8     hardware_adc
```

9	hardware_pio
10	Config
11	GUI
12	Fonts
13	OLED
14	SGP40
15	SHTC3
16	QMI8658
17	nec_receive_library)

4. Проаналізуйте, скомпілюйте та завантажте наведений код у Pico. В даному коді наведений приклад прийому і декодування сигналів ІЧ-пульта за протоколом NEC і виведення кодів кнопок на дисплей.

1	#include "OLED_1in5.h"
2	#include "GUI_Paint.h"
3	#include "DEV_Config.h"
4	#include "nec_receive.h"
5	
6	#define PIO_NUM (pio0)
7	#define IR_RX_PIN (15)
8	
9	int main()
10	{
11	uint8_t rx_address, rx_data;
12	PIO pio = PIO_NUM; // choose which PIO block to use (RP2040 has two: pio0 and
13	pio1)
14	uint8_t *BlackImage;
15	uint16_t Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH /
16	2) : (OLED_1in5_WIDTH / 2 + 1)) * OLED_1in5_HEIGHT;
17	
18	if (DEV_Module_Init() != 0)
19	{
20	return -1;
21	}
22	//Initialize the OLED
23	OLED_1in5_Init();
24	OLED_1in5_Clear(0);
25	if ((BlackImage = (uint8_t *)malloc(Imagesize)) == NULL)
26	{
27	while (1)
28	{
29	printf("Failed to apply for black memory...\r\n");
30	}

```

31     }
32     printf("Paint_NewImage\r\n");
33     Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0,
34 WHITE);
35     Paint_SetScale(16);
36     Paint_SelectImage(BlackImage);
37
38     int rx_sm = nec_rx_init(pio, IR_RX_PIN); // uses one state machine and 9
39 instructions
40     if (rx_sm == -1)
41     {
42         printf("could not configure PIO\n");
43         return -1;
44     }
45     printf("Hello,IR RX \r\n");
46     char str[20];
47     Paint_Clear(BLACK);
48     Paint_DrawString_EN(5, 10, "Wait receiving", &Font12, WHITE, BLACK);
49     OLED_1in5_Display(BlackImage);
50     while (true)
51     {
52         //wait receiving the infrared remote control signal
53         while (!pio_sm_is_rx_fifo_empty(pio, rx_sm))
54         {
55             uint32_t rx_frame = pio_sm_get(pio, rx_sm);
56             Paint_Clear(BLACK);
57             Paint_DrawString_EN(5, 10, "IR RX TEST", &Font12, WHITE, BLACK);
58
59             if (nec_decode_frame(rx_frame, &rx_address, &rx_data))
60             {
61                 sprintf(str, "received: %02x, %02x ", rx_address, rx_data);
62                 Paint_DrawString_EN(5, 25, str, &Font12, WHITE, BLACK);
63                 printf("received: %02x, %02x \r\n", rx_address, rx_data);
64             }
65             else
66             {
67                 sprintf(str, "received: %08x ", rx_frame);
68                 Paint_DrawString_EN(5, 25, str, &Font12, WHITE, BLACK);
69                 printf("received: %08x \r\n", rx_frame);
70             }
71             OLED_1in5_Display(BlackImage);
72         }
73         sleep_ms(10);
74     }
75 }

```

5. Створіть м'ютекс для доступу до спільних апаратних ресурсів (I²C шини).
6. Створіть черги для передачі даних від задач сенсорів до задачі відображення.
7. Створіть задачу, яка буде опитувати кнопку і записувати стан кнопки в чергу. Використайте `gpio_get()` для зчитування стану кнопки. Реалізуйте `debounce` (придушення брязкоту контактів). Передавайте значення `true/false` у чергу `xQueueButton`. Опитуйте кнопку кожні 50 мс.
8. Створіть окремі задачі для зчитування даних із сенсорів: температури та вологості (SHTC3), якості повітря (SGP40), акселерометра та гіроскопа (QMI8658). Кожна задача має: ініціалізувати свій сенсор, зчитувати дані з певним періодом (100мс), передавати дані у чергу.
9. Створіть задачі, які будуть зчитувати дані з потенціометра, фоторезистора та звукового сенсора і записувати у відповідні черги. Використайте ADC-канали GPIO26–GPIO28. Для кожного сенсора створіть окрему чергу (`xQueuePot`, `xQueueLight`, `xQueueSound`). Опитуйте сенсори кожні 50 мс.
10. Створіть задачу в якій буде реалізовано прийом і декодування сигналів ІЧ-пульта за протоколом NEC та передавання кодів кнопок через чергу.
11. Реалізуйте задачу виведення даних на OLED-дисплей, яка приймає дані з черг і відображає їх у зручному форматі. Оновлюйте екран з певною періодичністю (200-300 мс). Забезпечте перемикання режимів роботи системи або вибір відображуваних даних на дисплеї за допомогою ІЧ-пульта.
12. Додайте функцію вмикання/вимикання OLED-дисплея через кнопку. Якщо кнопка натиснута – перемикає стан змінної `display_on`. Якщо `display_on == false`, дисплей вимикається (очищується або переходить у `sleep`-режим).
13. Перевірте роботу системи та оцініть реакцію на команди користувача в реальному часі.